

CommerceHub-Plus テスト実装状況レポート

このプロジェクトにおいて実装されているテストについて、単体テスト、結合テスト、システムテストに分けて詳細に分析いたします。

🔍 テスト分類別詳細分析

1. 単体テスト (Unit Tests)

1.1 サーバーサイド ドメインレイヤー単体テスト

Order.test.ts (Orderドメインモデル)

テスト内容と手順	入力データ	期待出力	テスト結果	備考
プロパティ取得機能の検証	OrderProps (注文番号、商品情報、購入者情報等)	各プロパティが正しく取得される	✅ 成功	基本的なゲッター機能のテスト
支払い状態更新機能	新しい支払い状態 ("支払済")	支払い状態が正しく更新される	✅ 成功	状態遷移の正常系テスト
発送状態更新機能	新しい発送状態 ("発送済")	発送状態が正しく更新される	✅ 成功	状態遷移の正常系テスト
納品書データ生成機能	注文データ	正しいフォーマットの納品書データ	✅ 成功	ビジネスロジックのテスト
請求書データ生成機能	注文データ	正しいフォーマットの請求書データ	✅ 成功	ビジネスロジックのテスト
異常系: 不正支払い状態	不明な状態文字列	例外を投げずに状態を保存	✅ 成功	エラーハンドリングのテスト
異常系: 不正発送状態	不明な状態文字列	例外を投げずに状態を保存	✅ 成功	エラーハンドリングのテスト

OrderFactory.test.ts (注文ファクトリー)

テスト内容と手順	入力データ	期待出力	テスト結果	備考
ネット注文生成機能	商品情報、 購入者情報	正しいOrderインスタンス (クレジットカード、支払済)	✅ 成功	ファクトリーパターンのテスト
電話/FAX注文生成 (銀行振込)	商品情報、 購入者情報、 支払い方法	正しいOrderインスタンス (銀行振込、未払い)	✅ 成功	複数の支払い方法対応テスト
電話/FAX注文生成 (コンビニ決済)	商品情報、 購入者情報、 支払い方法	正しいOrderインスタンス (コンビニ決済、未払い)	✅ 成功	複数の支払い方法対応テスト
電話/FAX注文生成 (代金引換)	商品情報、 購入者情報、 支払い方法	正しいOrderインスタンス (代金引換、支払済)	✅ 成功	代金引換の特殊処理テスト

1.2 サーバーサイド アプリケーションレイヤー単体テスト

RegisterOrderService.test.ts（注文登録サービス）

テスト内容と手順	入力データ	期待出力	テスト結果	備考
単一商品のネット注文登録	有効な商品情報・購入者情報	注文番号の配列	✔️ 成功	モック使用の単体テスト
複数商品のネット注文登録	複数の商品情報	複数の注文番号の配列	✔️ 成功	バッチ処理のテスト
異常系: 商品が存在しない	存在しない商品番号	適切なエラーメッセージ	✔️ 成功	エラーハンドリングのテスト
異常系: 在庫不足	在庫を超える購入数量	適切なエラーメッセージ	✔️ 成功	ビジネスルール検証のテスト

AuthService.test.ts（認証サービス）

テスト内容と手順	入力データ	期待出力	テスト結果	備考
各ロールの認証成功	正しいロールとパスワード	true	✔️ 成功	複数のロール対応テスト
認証失敗	間違ったパスワード	false	✔️ 成功	セキュリティテスト
認証が必要な操作の判定	各種ロール	適切な認証要否判定	✔️ 成功	権限管理のテスト

1.3 サーバーサイド インフラストラクチャレイヤー単体テスト

OrderRepository.test.ts（注文リポジトリ）

テスト内容と手順	入力データ	期待出力	テスト結果	備考
注文の保存機能	Orderオブジェクト	正常に保存完了	✔️ 成功	データアクセス層のテスト
注文番号による検索	注文番号	該当するOrderオブジェクト	✔️ 成功	検索機能のテスト

1.4 フロントエンド 単体テスト

useAuth.test.tsx（認証フック）

テスト内容と手順	入力データ	期待出力	テスト結果	備考
AuthProvider内での正常使用	AuthProviderでラップ	認証コンテキストの取得	✔️ 成功	Reactフックのテスト
初期状態での未認証	なし	role: null, isAuthenticated: false	✔️ 成功	初期化テスト
localStorage復元機能	localStorage内のロール情報	保存されたロールの復元	✔️ 成功	永続化機能のテスト
AuthProvider外での使用	AuthProviderなし	エラーメッセージ	✔️ 成功	エラーハンドリングのテスト

useCart.test.ts（カートフック）

テスト内容と手順	入力データ	期待出力	テスト結果	備考
商品追加機能	商品情報と数量	カートに商品が追加される	✔️ 成功	カート管理のテスト
商品削除機能	商品番号	カートから商品が削除される	✔️ 成功	カート管理のテスト
数量更新機能	商品番号と新しい数量	商品数量が更新される	✔️ 成功	カート管理のテスト
カートクリア機能	なし	カートが空になる	✔️ 成功	カート管理のテスト

useProducts.test.ts（商品管理フック）

テスト内容と手順	入力データ	期待出力	テスト結果	備考
商品検索機能	検索条件（商品番号・商品名）	該当商品の取得	✔️ 成功	API連携のテスト
ネットワークエラー時の処理	不正なリクエスト	エラーメッセージの設定	✔️ 成功	エラーハンドリングのテスト
ローディング状態管理	API呼び出し中	ローディング状態の管理	✔️ 成功	状態管理のテスト

2. 結合テスト（Integration Tests）

2.1 サーバーサイド API結合テスト

orderController.test.ts（注文API結合テスト）

テスト内容と手順	入力データ	期待出力	テスト結果	備考
ネット注文API（正常系）	有効な注文データ	200レスポンスと注文番号	✔️ 成功	API経由の注文処理テスト
ネット注文API （バリデーションエラー）	不正なリクエストデータ	400エラーレスポンス	✔️ 成功	入力検証のテスト
ネット注文API （在庫不足エラー）	在庫を超える注文	400エラーと適切なメッセージ	✔️ 成功	ビジネスルール検証のテスト
支払い状態更新API（正常系）	注文番号と新しい状態	200成功レスポンス	✔️ 成功	状態更新APIのテスト
支払い状態更新API（異常系）	status未指定	400エラーレスポンス	✔️ 成功	パラメータ検証のテスト
納品書取得API（正常系）	注文番号	納品書データの取得	✔️ 成功	ドキュメント生成APIのテスト
納品書取得API（異常系）	存在しない注文番号	エラーレスポンス	✔️ 成功	エラーハンドリングのテスト

authController.test.ts（認証API結合テスト）

テスト内容と手順	入力データ	期待出力	テスト結果	備考
認証API（正常系）	正しいロールとパスワード	200成功レスポンス	✔️ 成功	認証フローのテスト
認証API（認証失敗）	間違ったパスワード	401認証エラー	✔️ 成功	セキュリティテスト
認証API（パラメータ不足）	ロールまたはパスワード未指定	400バリデーションエラー	✔️ 成功	入力検証のテスト
認証API（無効ロール）	存在しないロール	400エラーレスポンス	✔️ 成功	ロール検証のテスト
ロール一覧取得API	なし	認証が必要なロール一覧	✔️ 成功	設定情報APIのテスト

productsController.test.ts（商品API結合テスト）

テスト内容と手順	入力データ	期待出力	テスト結果	備考
商品検索API	検索条件	商品情報の取得	✔️ 成功	検索APIのテスト

2.2 フロントエンド コンポーネント結合テスト

Login.test.tsx（ログインコンポーネント結合テスト）

テスト内容と手順	入力データ	期待出力	テスト結果	備考
ロール選択とUI表示	ロール選択操作	適切なUI表示 (パスワードフィールドの表示・非表示)	✔️ 成功	UIロジック結合テスト
購入者ログイン (認証なし)	購入者ロール選択	パスワード認証なしでログイン成功	✔️ 成功	業務フロー結合テスト
職員ログイン（認証あり）	職員ロールとパスワード	API呼び出し後にログイン成功	✔️ 成功	認証フロー結合テスト
ロール変更時のリセット	ロール変更操作	パスワードとエラーのクリア	✔️ 成功	状態管理結合テスト

各ダッシュボードコンポーネント結合テスト

コンポーネント	テスト内容と手順	入力データ	期待出力	テスト結果	備考
AccountantDashboard	注文検索とデータ表示	検索条件	注文データの一覧表示	✔️ 成功	検索機能とUI結合テスト
OrderTakerDashboard	商品検索と注文登録	商品検索条件、 注文情報	商品表示と注文処理	✔️ 成功	注文業務フロー結合テスト
PurchaserDashboard	カート操作と注文処理	商品選択、 カート操作	カート状態変更と注文完了	✔️ 成功	購入フロー結合テスト
ShippingDashboard	発送業務と状態更新	発送操作	発送状態の更新	✔️ 成功	発送業務フロー結合テスト

3. システムテスト（System Tests）

3.1 サーバー全体システムテスト

index.test.ts（サーバー全体テスト）

テスト内容と手順	入力データ	期待出力	テスト結果	備考
サーバー起動確認	ルートパスへのGETリクエスト	200レスポンスと起動メッセージ	✔️ 成功	サーバー基本動作テスト
商品検索エンドポイント (商品番号)	商品番号での検索クエリ	該当商品データの取得	✔️ 成功	エンドツーエンド検索テスト
商品検索エンドポイント (商品名)	商品名での検索クエリ	該当商品データの取得	✔️ 成功	エンドツーエンド検索テスト

3.2 フロントエンド全体システムテスト

App.test.tsx（アプリケーション全体テスト）

テスト内容と手順	入力データ	期待出力	テスト結果	備考
初期状態での画面表示	アプリケーション起動	ログイン画面の表示	✔️ 成功	アプリ初期化テスト
未認証時のルートガード	保護されたページへのアクセス	ログイン画面へのリダイレクト	✔️ 成功	セキュリティ機能テスト
認証状態の永続化	ログイン後のページリロード	ダッシュボードの維持	✔️ 成功	状態永続化テスト
ログアウト機能	ログアウトボタン操作	ログイン画面への遷移	✔️ 成功	認証フロー全体テスト